More Python

Name: Rahul Vedant Pemsing
Student ID: 36305529
Date: 01/10/2028

Learning Journey and Evidence

Prior Knowledge and Experience

Before taking FIT1045, I had already learned Python through online tutorials and YouTube guides. This allowed me to reach a level of proficiency comparable to my A-level programming language, Visual Basic (console).

Before attempting this task, I had already built a strong foundation through both my previous studies and the earlier tasks (P1–P8).

Through P1–P11, I further developed my programming knowledge step by step:

- P1 (Learning Plan): Helped me set clear goals, organize my study approach, and identify areas where I needed to improve.
- P2 (Building and Running): Gave me practical skills of executing programs, as well as debugging common errors.
- P3 (Sequence and Data): Strengthened my understanding of variables, operators, and data handling, which is the basis of problem-solving experience.
- P4 (Control Flow): Taught me how to use conditionals and loops effectively to control the execution path of programs.
- P5 (C/C++): Reinforced syntax rules and highlighted the importance of precision in coding (e.g., C++ being case-sensitive).
- P6 (Structuring Code): Showed me how to modularize programs using functions and procedures, improving readability and reusability.
- P7 (Structuring Data): Introduced enums, structures, and the organization of data in memory, preparing me for more advanced concepts like pointers and references.
- P8 (Working with Multiples): Understood how to use array in program
- P9 (Another language Python): Refresh my knowledge about the python language
- P10 (Classes and Object): Refresh my knowledge about OOP in python

- P11 (Inheritance and Polymorthism): Refresh my knowledge about more complex OOP in python

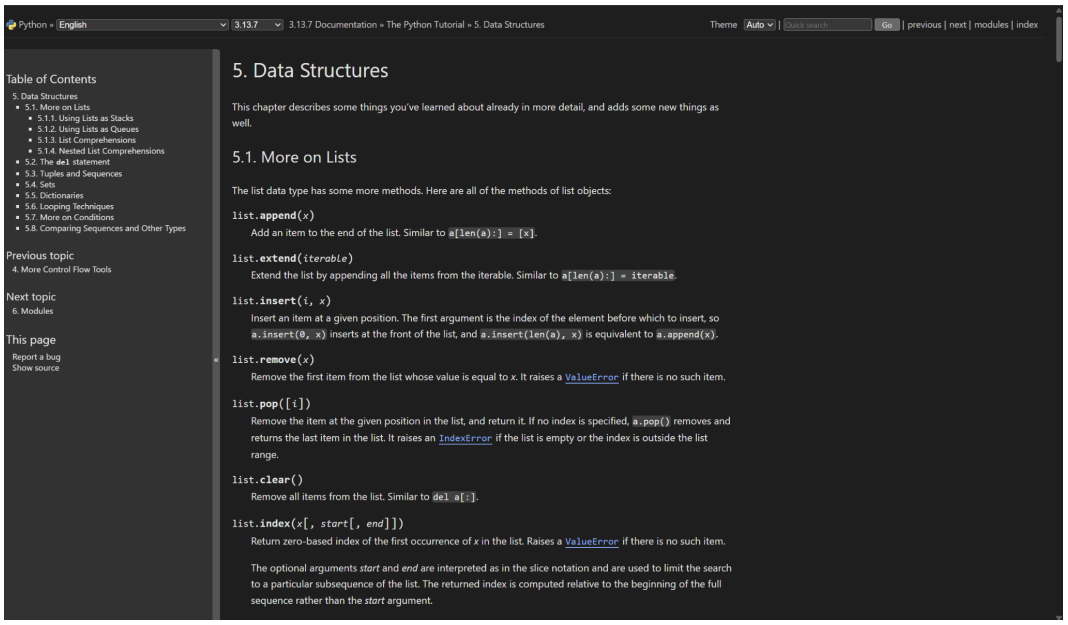
Additionally, I am currently taking the unit FIT1056:Introduction to Software Engineering, which extensively uses python. Thus this also helped deepen my knowledge of python programming.

Study to Acquire, Refresh, and/or Extend Capability

To study and master this topic I did the following:

30/09/2025 - Read the python guide extensively

Read about list :



Python » English 3.13.7 3.13.7 Documentation » The Python Tutorial » 5. Data Structures

Theme Auto | Quick search Go | previous | next | modules | index

Table of Contents

5. Data Structures
- 5.1. More on Lists
  - 5.1.1. Using Lists as Stacks
  - 5.1.2. Using Lists as Queues
  - 5.1.3. List Comprehensions
  - 5.1.4. Nested List Comprehensions
- 5.2. The del statement
- 5.3. Tuples and Sequences
- 5.4. Sets
- 5.5. Dictionaries
- 5.6. Looping Techniques
- 5.7. More on Conditions
- 5.8. Comparing Sequences and Other Types

Previous topic
4. More Control Flow Tools

Next topic
6. Modules

This page
Report a bug
Show source

5. Data Structures

This chapter describes some things you've learned about already in more detail, and adds some new things as well.

5.1. More on Lists

The list data type has some more methods. Here are all of the methods of list objects:

`list.append(x)`
Add an item to the end of the list. Similar to `a[len(a):] = [x]`.

`list.extend(iterable)`
Extend the list by appending all the items from the iterable. Similar to `a[len(a):] = iterable`.

`list.insert(i, x)`
Insert an item at a given position. The first argument is the index of the element before which to insert, so `a.insert(0, x)` inserts at the front of the list, and `a.insert(len(a), x)` is equivalent to `a.append(x)`.

`list.remove(x)`
Remove the first item from the list whose value is equal to *x*. It raises a `ValueError` if there is no such item.

`list.pop([i])`
Remove the item at the given position in the list, and return it. If no index is specified, `a.pop()` removes and returns the last item in the list. It raises an `IndexError` if the list is empty or the index is outside the list range.

`list.clear()`
Remove all items from the list. Similar to `del a[:]`.

`list.index(x[, start[, end]])`
Return zero-based index of the first occurrence of *x* in the list. Raises a `ValueError` if there is no such item.

The optional arguments *start* and *end* are interpreted as in the slice notation and are used to limit the search to a particular subsequence of the list. The returned index is computed relative to the beginning of the full sequence rather than the *start* argument.

Read about Dictionaries:

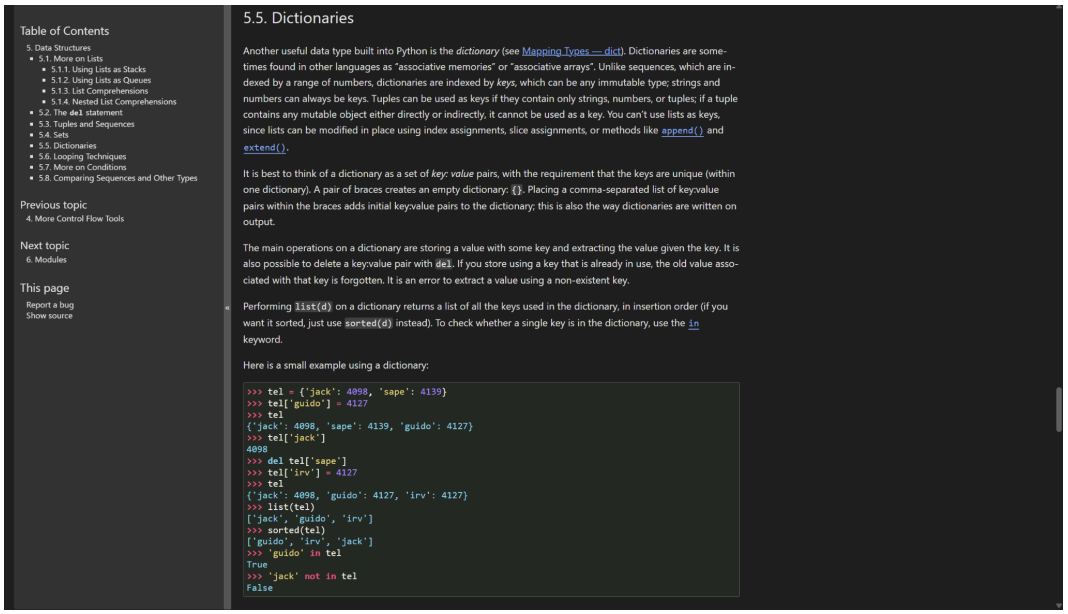


Table of Contents

5. Data Structures
- 5.1. More on Lists
  - 5.1.1. Using Lists as Stacks
  - 5.1.2. Using Lists as Queues
  - 5.1.3. List Comprehensions
  - 5.1.4. Nested List Comprehensions
- 5.2. The del statement
- 5.3. Tuples and Sequences
- 5.4. Sets
- 5.5. Dictionaries
- 5.6. Looping Techniques
- 5.7. More on Conditions
- 5.8. Comparing Sequences and Other Types

Previous topic
4. More Control Flow Tools

Next topic
6. Modules

This page
Report a bug
Show source

5.5. Dictionaries

Another useful data type built into Python is the *dictionary* (see Mapping Types — dict). Dictionaries are sometimes found in other languages as "associative memories" or "associative arrays". Unlike sequences, which are indexed by a range of numbers, dictionaries are indexed by *keys*, which can be any immutable type; strings and numbers can always be keys. Tuples can be used as keys if they contain only strings, numbers, or tuples; if a tuple contains any mutable object either directly or indirectly, it cannot be used as a key. You can't use lists as keys, since lists can be modified in place using index assignments, slice assignments, or methods like `append()` and `extend()`.

It is best to think of a dictionary as a set of *key: value* pairs, with the requirement that the keys are unique (within one dictionary). A pair of braces creates an empty dictionary: `{}`. Placing a comma-separated list of key:value pairs within the braces adds initial key:value pairs to the dictionary; this is also the way dictionaries are written on output.

The main operations on a dictionary are storing a value with some key and extracting the value given the key. It is also possible to delete a key:value pair with `del`. If you store using a key that is already in use, the old value associated with that key is forgotten. It is an error to extract a value using a non-existent key.

Performing `list(d)` on a dictionary returns a list of all the keys used in the dictionary, in insertion order (if you want it sorted, just use `sorted(d)` instead). To check whether a single key is in the dictionary, use the `in` keyword.

Here is a small example using a dictionary:

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']
4098
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)
['jack', 'guido', 'irv']
>>> sorted(tel)
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> 'jack' not in tel
False
```

Read about Looping techniques:

Table of Contents

5. Data Structures
- 5.1. More on Lists
  - 5.1.1. Using Lists as Stacks
  - 5.1.2. Using Lists as Queues
  - 5.1.3. List Comprehensions
  - 5.1.4. Nested List Comprehensions
- 5.2. The **del** statement
- 5.3. Tuples and Sequences
- 5.4. Sets
- 5.5. Dictionaries
- 5.6. Looping Techniques
- 5.7. More on Conditions
- 5.8. Comparing Sequences and Other Types

Previous topic
4. More Control Flow Tools

Next topic
6. Modules

This page
Report a bug
Show source

5.6. Looping Techniques

When looping through dictionaries, the key and corresponding value can be retrieved at the same time using the `items()` method.

```
>>> knights = {'gallahad': 'the pure', 'robin': 'the brave'}
>>> for k, v in knights.items():
...     print(k, v)
...
gallahad the pure
robin the brave
```

When looping through a sequence, the position index and corresponding value can be retrieved at the same time using the `enumerate()` function.

```
>>> for i, v in enumerate(['tic', 'tac', 'toe']):
...     print(i, v)
...
0 tic
1 tac
2 toe
```

To loop over two or more sequences at the same time, the entries can be paired with the `zip()` function.

```
>>> questions = ['name', 'quest', 'favorite color']
>>> answers = ['lancelot', 'the holy grail', 'blue']
>>> for q, a in zip(questions, answers):
...     print('What is your {0}?  It is {1}.'.format(q, a))
...
What is your name?  It is lancelot.
What is your quest?  It is the holy grail.
What is your favorite color?  It is blue.
```

To loop over a sequence in reverse, first specify the sequence in a forward direction and then call the `reversed()` function.

```
>>> for i in reversed(range(1, 10, 2)):
...     print(i)
...
9
7
5
3
1
```

To loop over a sequence in sorted order, use the `sorted()` function which returns a new sorted list while leaving

Read about formatting functions:

Python » English 3.13.7 3.13.7 Documentation » The Python Tutorial » 7. Input and Output

Theme Auto | Quick search Go | previous | next | modules | index

Table of Contents

7. Input and Output
- 7.1. Fancier Output Formatting
  - 7.1.1. Formatted String Literals
  - 7.1.2. The String format() Method
  - 7.1.3. Manual String Formatting
  - 7.1.4. Old string formatting
- 7.2. Reading and Writing Files
  - 7.2.1. Methods of File Objects
  - 7.2.2. Saving structured data with **json**

Previous topic
6. Modules

Next topic
8. Errors and Exceptions

This page
Report a bug
Show source

7. Input and Output

There are several ways to present the output of a program; data can be printed in a human-readable form, or written to a file for future use. This chapter will discuss some of the possibilities.

7.1. Fancier Output Formatting

So far we've encountered two ways of writing values: *expression statements* and the `print()` function. (A third way is using the `write()` method of file objects; the standard output file can be referenced as `sys.stdout`. See the Library Reference for more information on this.)

Often you'll want more control over the formatting of your output than simply printing space-separated values. There are several ways to format output.

- To use formatted string literals, begin a string with `f` or `F` before the opening quotation mark or triple quotation mark. Inside this string, you can write a Python expression between `{` and `}` characters that can refer to variables or literal values.

```
>>> year = 2016
>>> event = 'Referendum'
>>> f'Results of the {year} {event}'
'Results of the 2016 Referendum'
```

- The `str.format()` method of strings requires more manual effort. You'll still use `{` and `}` to mark where a variable will be substituted and can provide detailed formatting directives, but you'll also need to provide the information to be formatted. In the following code block there are two examples of how to format variables:

```
>>> yes_votes = 42_572_654
>>> total_votes = 85_705_149
>>> percentage = yes_votes / total_votes
>>> '{:-9} YES votes  {:2.2%}'.format(yes_votes, percentage)
' 42572654 YES votes  49.67%'
```

Notice how the `yes_votes` are padded with spaces and a negative sign only for negative numbers. The example also prints `percentage` multiplied by 100, with 2 decimal places and followed by a percent sign (see Format Specification Mini-Language for details).

Read about Text file and JSON file:

Table of Contents
7. Input and Output
- 7.1. Fancier Output Formatting
  - 7.1.1. Formatted String Literals
  - 7.1.2. The String format() Method
  - 7.1.3. Manual String Formatting
  - 7.1.4. Old string formatting
- 7.2. Reading and Writing Files
  - 7.2.1. Methods of File Objects
  - 7.2.2. Saving structured data with `json`

Previous topic
6. Modules

Next topic
8. Errors and Exceptions

This page
Report a bug
Show source

7.2. Reading and Writing Files

open() returns a file object, and is most commonly used with two positional arguments and one keyword argument: `open(filename, mode, encoding=None)`

```
>>> f = open('workfile', 'w', encoding="utf-8")
```

The first argument is a string containing the filename. The second argument is another string containing a few characters describing the way in which the file will be used. *mode* can be `'r'` when the file will only be read, `'w'` for only writing (an existing file with the same name will be erased), and `'a'` opens the file for appending; any data written to the file is automatically added to the end. `'r+'` opens the file for both reading and writing. The *mode* argument is optional; `'r'` will be assumed if it's omitted.

Normally, files are opened in *text mode*, that means, you read and write strings from and to the file, which are encoded in a specific *encoding*. If *encoding* is not specified, the default is platform dependent (see open()). Because UTF-8 is the modern de-facto standard, `encoding="utf-8"` is recommended unless you know that you need to use a different encoding. Appending a `'b'` to the mode opens the file in *binary mode*. Binary mode data is read and written as bytes objects. You can not specify *encoding* when opening file in binary mode.

In text mode, the default when reading is to convert platform-specific line endings (`\n` on Unix, `\r\n` on Windows) to just `\n`. When writing in text mode, the default is to convert occurrences of `\n` back to platform-specific line endings. This behind-the-scenes modification to file data is fine for text files, but will corrupt binary data like that in `JPEG` or `EXE` files. Be very careful to use binary mode when reading and writing such files.

It is good practice to use the with keyword when dealing with file objects. The advantage is that the file is properly closed after its suite finishes, even if an exception is raised at some point. Using `with` is also much shorter than writing equivalent try-finally blocks:

```
>>> with open('workfile', encoding="utf-8") as f:
...     read_data = f.read()

>>> # We can check that the file has been automatically closed.
>>> f.closed
True
```

If you're not using the with keyword, then you should call `f.close()` to close the file and immediately free up any system resources used by it.

Warning: Calling `f.write()` without using the `with` keyword or calling `f.close()` **might** result in the arguments of `f.write()` not being completely written to the disk, even if the program exits successfully.

Read about CSV file:

Table of Contents
csv — CSV File Reading and Writing
- Module Contents
- Dialects and Formatting Parameters
- Reader Objects
- Writer Objects
- Examples

Previous topic
File Formats

Next topic
configparser — Configuration file parser

This page
Report a bug
Show source

csv — CSV File Reading and Writing

Source code: Lib/csv.py

The so-called CSV (Comma Separated Values) format is the most common import and export format for spreadsheets and databases. CSV format was used for many years prior to attempts to describe the format in a standardized way in RFC 4180. The lack of a well-defined standard means that subtle differences often exist in the data produced and consumed by different applications. These differences can make it annoying to process CSV files from multiple sources. Still, while the delimiters and quoting characters vary, the overall format is similar enough that it is possible to write a single module which can efficiently manipulate such data, hiding the details of reading and writing the data from the programmer.

The csv module implements classes to read and write tabular data in CSV format. It allows programmers to say, "write this data in the format preferred by Excel," or "read data from this file which was generated by Excel," without knowing the precise details of the CSV format used by Excel. Programmers can also describe the CSV formats understood by other applications or define their own special-purpose CSV formats.

The csv module's reader and writer objects read and write sequences. Programmers can also read and write data in dictionary form using the DictReader and DictWriter classes.

See also:

PEP 305 - CSV File API
The Python Enhancement Proposal which proposed this addition to Python.

Module Contents

The csv module defines the following functions:

`csv.reader(csvfile, /, dialect='excel', **fmtparams)`
Return a reader object that will process lines from the given *csvfile*. A csvfile must be an iterable of strings, each in the reader's defined csv format. A csvfile is most commonly a file-like object or list. If *csvfile* is a file object, it should be opened with `newline=''`. [1] An optional *dialect* parameter can be given which is used to define a set of parameters specific to a particular CSV dialect. It may be an instance of a subclass of the

What I learned about Lists:

- Learned that lists in Python are ordered, mutable collections that can store elements of different data types.
- Practiced common operations such as indexing, slicing, appending, inserting, and removing items.
- Understood list methods like .sort(), .reverse(), and .extend().
- Learned about list comprehensions as a concise way to build new lists based on conditions or transformations.

- Realized how versatile lists are for managing sequences of data, such as storing student grades or names.

What I learned about Dictionaries:

- Understood that dictionaries store data as key-value pairs, allowing for fast lookups.
- Learned how to create dictionaries, add new keys, update values, and delete items.
- Practiced iterating through keys, values, and key-value pairs with .keys(), .values(), and .items().
- Saw how dictionaries are useful for real-world data storage, e.g., keeping track of student details like { "name": "Alex", "grade": 85 }.
- Appreciated that dictionaries allow flexible, descriptive data structures compared to lists.

What I learned about Looping Techniques:

- Learned advanced ways to loop, such as using enumerate() to access both index and value.
- Used .items() in dictionaries to loop over key-value pairs directly.
- Practiced looping with conditions, list comprehensions, and nested loops for structured data.
- Realized how these techniques reduce the need for manual counters and make code cleaner and more Pythonic.

What I learned about Formatting Functions:

- Learned how to use string formatting to present data clearly (f-strings, .format(), and % formatting).
- Practiced aligning text, controlling decimal places, and adding padding or width adjustments in formatted output.
- Understood how formatting functions make data presentation more professional, e.g., printing a grade report in neat columns.
- Discovered that formatting can also help in logging and debugging by showing variable values clearly.

What I learned about Text File and JSON File:

- Learned how to open, read, and write text files using open(), .read(), .write(), and .close().
- Practiced using context managers (with open(...) as f) to handle files safely.
- Learned about JSON format as a way to represent structured data in text form (key-value pairs similar to dictionaries).
- Used the json module to load data from a JSON file into a Python dictionary and to write Python data into a JSON file.
- Understood how these skills are useful in saving and loading data between sessions, such as storing student logs.

What I learned about CSV File:

- Learned that CSV files store tabular data separated by commas.
- Used Python's built-in csv module to read rows into lists or dictionaries.
- Practiced writing structured data into CSV files for later analysis.
- Saw how CSV is commonly used in data science, spreadsheets, and databases.
- Understood the importance of CSV handling for real-world projects, such as exporting student statistics or grade logs.

30/09/2025 → 01/10/2025 - Coding [A Python program that encompasses all my python knowledge]

Description:

Created a simple UI student logging system in python using streamlit for the UI.

My code:

C4.py

C4.py > load_students

```python
import streamlit as st
import csv
import unittest
import io
import os
import statistics


STUDENT_FILE = "students.csv"   #file path, create it if not found.

def init_student_file():
    """Ensure the student CSV file exists with headers."""
    if not os.path.exists(STUDENT_FILE):
        with open(STUDENT_FILE, "w", newline="") as f:
            writer = csv.DictWriter(f, fieldnames=["name", "age", "grade"])
            writer.writeheader()


def load_students():
    """Load students from the constant CSV file."""
    students = []
    init_student_file()
    with open(STUDENT_FILE, "r") as f:
        reader = csv.DictReader(f)
        for row in reader:
            students.append({"name": row["name"], "age": int(row["age"]), "grade": float(row["grade"])})
    return students


def save_student(student):
    """Append a single student to the CSV file."""
    init_student_file()
    with open(STUDENT_FILE, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "age", "grade"])
        writer.writerow(student)


def format_student(student):
    """Return a nicely formatted string for a student dict."""
    return f"👤 {student['name']:<15} | Age: {student['age']:<2} | Grade: {student['grade']}%"


def add_student(students, name, age, grade):
    """Add a new student to the list and save to file."""
    student = {"name": name, "age": int(age), "grade": float(grade)}
    students.append(student)
    save_student(student)
    return students
```

```python

def compute_stats(students):
    """Compute average, highest and lowest grade student."""
    if not students:
        return None, None, None

    avg = statistics.mean([s["grade"] for s in students])
    top_student = max(students, key=lambda s: s["grade"])
    bottom_student = min(students, key=lambda s: s["grade"])
    return avg, top_student, bottom_student


#class testing- it would have been it own sperate file if the file submission was not limited to 1.
class TestStudentLog(unittest.TestCase):

    def test_add_student(self):
        students = []
        updated = add_student(students, "Alice", 20, 90)
        self.assertEqual(updated[-1]["name"], "Alice")

    def test_format_student(self):
        student = {"name": "Bob", "age": 18, "grade": 75}
        formatted = format_student(student)
        self.assertIn("Bob", formatted)
        self.assertIn("75", formatted)


def run_tests():
    """Run unittest inside the Streamlit app and return results as a string."""
    buffer = io.StringIO()
    runner = unittest.TextTestRunner(stream=buffer, verbosity=2)
    suite = unittest.TestLoader().loadTestsFromTestCase(TestStudentLog)
    runner.run(suite)
    return buffer.getvalue()

```

```python
#UI start here, usually it would be in it own file but since the file submission is limited to 1, I have included it here.
st.title("📘 Student Log Application")

# Sidebar for options
st.sidebar.header("Options")
menu = st.sidebar.radio("Choose Action", ["Student Log", "Run Unit Tests"])

if menu == "Student Log":
    # Initialize students from file
    if "students" not in st.session_state:
        st.session_state.students = load_students()

    # Create tabs
    tab1, tab2 = st.tabs(["📝 Student List", "📊 Stats"])

    with tab1:
        # Add new student
        st.subheader("➕ Add a New Student")
        with st.form("student_form"):
            name = st.text_input("Name")
            age = st.number_input("Age", min_value=5, max_value=100, step=1)
            grade = st.number_input("Grade (%)", min_value=0, max_value=100, step=1)
            submitted = st.form_submit_button("Add Student")

            if submitted and name:
                st.session_state.students = add_student(st.session_state.students, name, age, grade)
                st.success(f"Student {name} added and saved!")

        # Display students
        st.subheader("📋 Student List")
        if st.session_state.students:
            for s in st.session_state.students:
                st.write(format_student(s))
        else:
            st.info("No students yet. Add some!")

        # Search
        st.subheader("🔍 Search Student")
        search_name = st.text_input("Enter name to search:")
        if search_name:
            results = [s for s in st.session_state.students if search_name.lower() in s["name"].lower()]
            if results:
                st.success("Found:")
                for s in results:
                    st.write(format_student(s))
            else:
                st.error("No student found.")
```

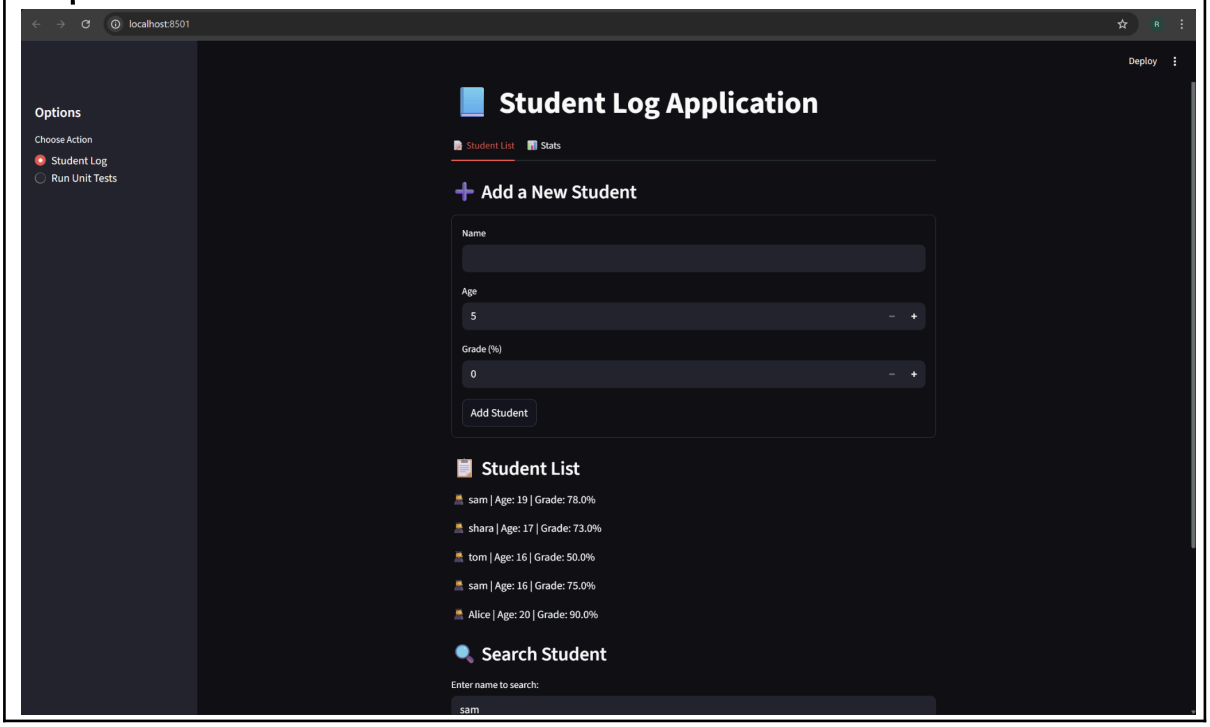
```python
    with tab2:
        st.subheader("📊 Student Statistics")
        avg, top, bottom = compute_stats(st.session_state.students)
        if avg is None:
            st.info("No data available.")
        else:
            st.write(f"📈 **Average Grade:** {avg:.2f}%")
            st.write(f"🥇 **Top Student:** {top['name']} ({top['grade']}%)")
            st.write(f"🥉 **Lowest Student:** {bottom['name']} ({bottom['grade']}%)")

elif menu == "Run Unit Tests":
    st.subheader("🧪 Running Unit Tests")
    result_text = run_tests()
    st.text(result_text)

```

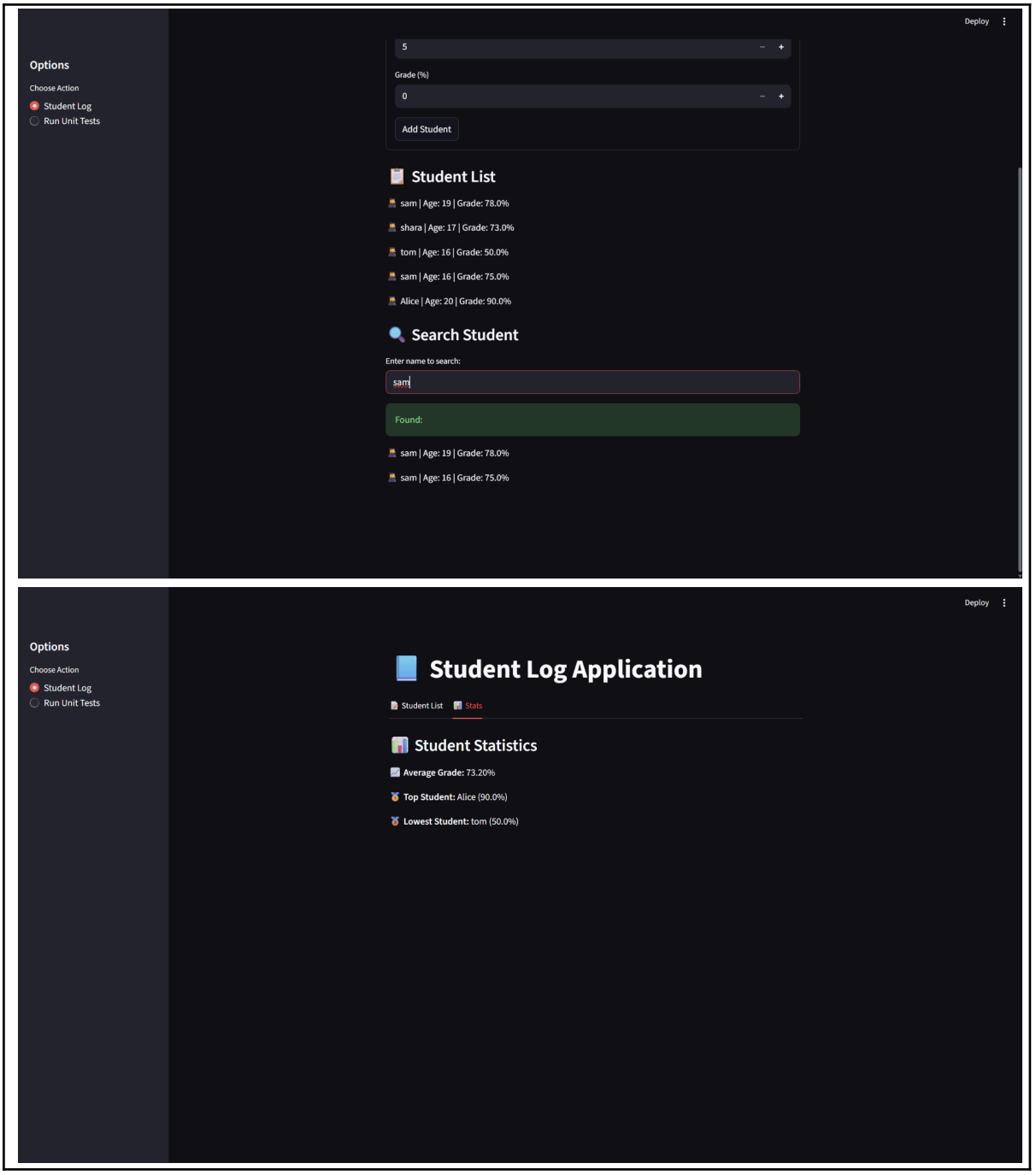
How to run run:

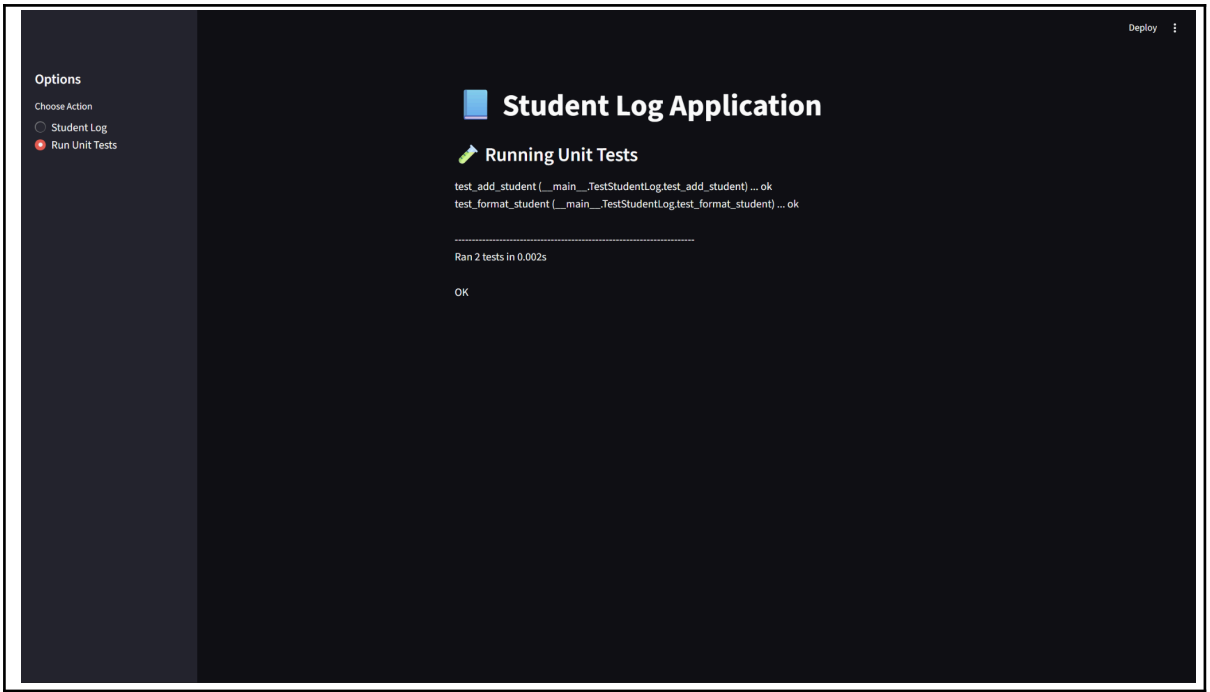
```
Rahul@VRP_Legion MINGW64 /c/Users/Rahul/Desktop/mycode/my_python_code/C4
$ source C:/Users/Rahul/Desktop/mycode/my_python_code/C4/venv/Scripts/activate
(venv)
Rahul@VRP_Legion MINGW64 /c/Users/Rahul/Desktop/mycode/my_python_code/C4
$ streamlit run C4.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://10.192.56.214:8501
```

The program is run as shown(streamlit run C4.py)
Also, I ran a virtual environment for C4 so streamlit and python ver3.13.7 doesn't mess with the global python install which doesn't have streamlit and is running on older version

Output:




Deploy
Options
Choose Action
Student Log
Run Unit Tests
5
Grade (%)
0
Add Student
Student List
sam | Age: 19 | Grade: 78.0%
shara | Age: 17 | Grade: 73.0%
tom | Age: 16 | Grade: 50.0%
sam | Age: 16 | Grade: 75.0%
Alice | Age: 20 | Grade: 90.0%
Search Student
Enter name to search:
sam
Found:
sam | Age: 19 | Grade: 78.0%
sam | Age: 16 | Grade: 75.0%
Deploy
Options
Choose Action
Student Log
Run Unit Tests
Student Log Application
Student List
Stats
Student Statistics
Average Grade: 73.20%
Top Student: Alice (90.0%)
Lowest Student: tom (50.0%)

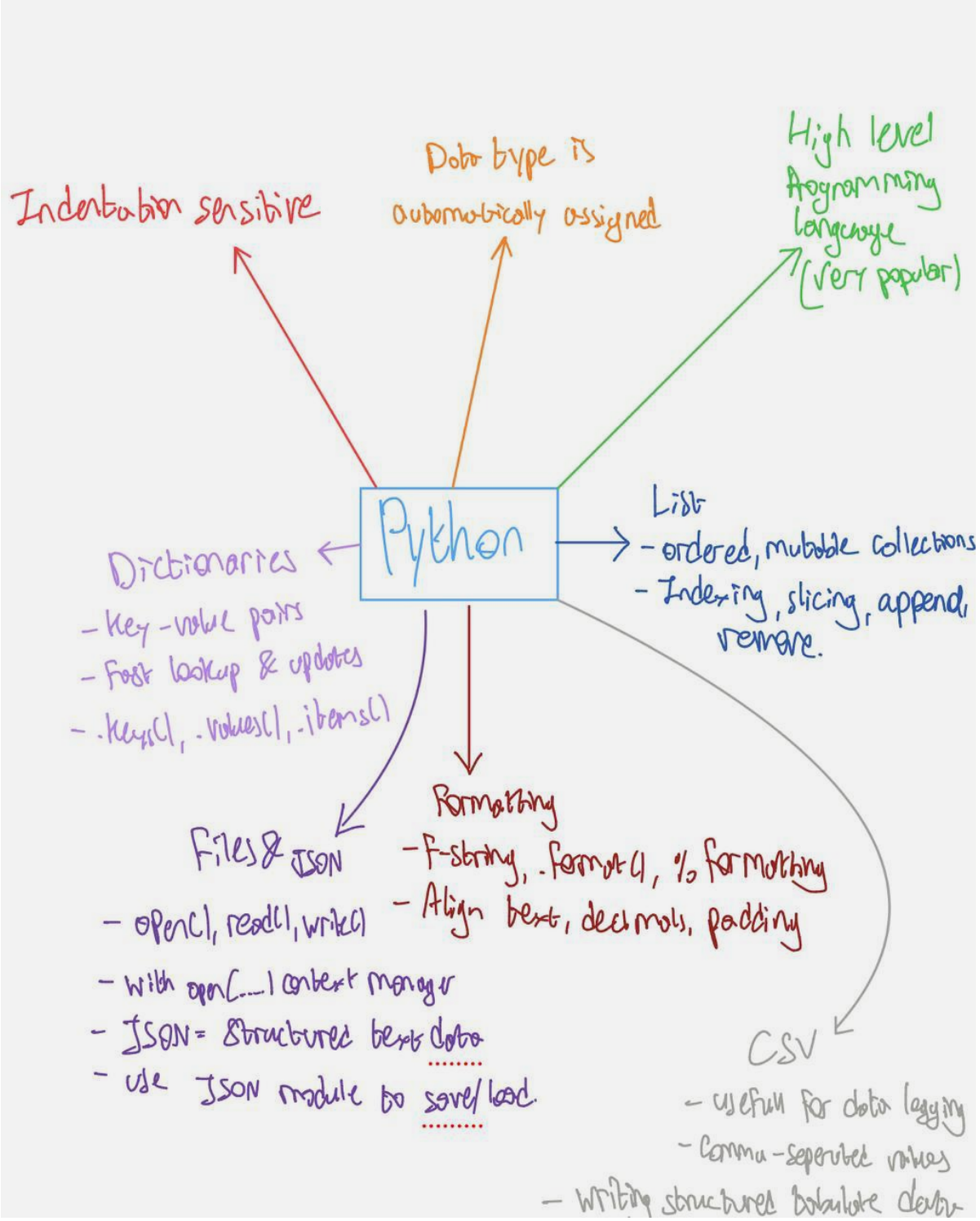


From building the student logging system with Streamlit, I learned how to connect theoretical Python knowledge with practical application. Specifically:

- File Handling Integration: I applied what I learned about text files, JSON, and CSV to store and retrieve student data reliably. This made me realize how important persistent data storage is for real-world applications.
- Object-Oriented Programming (OOP): I reinforced my understanding of classes, inheritance, and polymorphism by designing structured components that managed students and their grades.
- Data Formatting and Presentation: By using string formatting, I was able to create readable output, such as grade reports and summaries. This improved the usability of the program.
- Looping and Data Structures: Lists and dictionaries were central to storing, searching, and updating student information. This showed me how versatile Python's core data types are when solving practical problems.
- UI with Streamlit: I learned how to create a simple user interface to interact with my program. This was my first experience combining backend logic with a frontend tool, which helped me understand the importance of user experience in software development.
- Environment Management: Running the program in a virtual environment taught me good practices in managing dependencies, ensuring that my project ran smoothly without affecting global Python installations.

Overall, this coding exercise gave me a much deeper appreciation for how Python concepts connect together to build a working application rather than just isolated exercises.

My Python Notes



Reflection

What is the most important thing you learned from this and why?

The most important thing I learned was how to combine different Python concepts; such as file handling, data structures, and OOP; into one cohesive project. This is important because in real-world programming, skills are rarely applied in isolation; instead, they must integrate seamlessly to solve practical problems.

What strategies did you apply when extending your Python capability?

I used a combination of structured reading (following the Python guide), hands-on practice (writing small test snippets before implementing them in the main project), and iterative improvement (testing the program step by step). I also relied on breaking down the task into smaller parts, such as first handling file storage before moving on to building the UI.

How did your understanding of the programming concepts help you with this task?

My understanding of inheritance and polymorphism helped me design reusable and extendable components, while knowledge of loops, lists, and dictionaries made managing student data efficient. My familiarity with debugging strategies also meant I could quickly identify and fix logic errors when integrating different parts of the program.

What will you do differently/the same when you have to learn your next programming language?

Next time, I will continue using the same strategy of combining reading with practical coding projects, as this made my learning more effective. However, I will also focus more on documenting my process from the start, as keeping clear notes would make it easier to track my progress and revisit tricky concepts.

References

Generative AI acknowledgement

No AI was used for either the code or this document.